Coverage for **install/environment_model/lib/python3.10/site-packages/environment_model/environment_model.py**: 99%

124 statements | 123 run | 1 missing | 0 excluded

« prev ^ index » next coverage.py v7.11.0, created at 2025-11-09 02:30 +0100

```python
# import required libraries
import rclpy
from rclpy.node import Node
from tf2_ros import TransformBroadcaster, TransformException
import tf_transformations
from tf2_geometry_msgs import do_transform_point
from geometry_msgs.msg import PointStamped
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener
from sklearn.cluster import DBSCAN
import numpy as np

# import required standard and custom data types
from sensor_msgs.msg import Image
from nav_msgs.msg import Odometry
from vision_msgs.msg import Detection2DArray
from custom_msgs.msg import DetectedObject, DetectedObjectsPositionArray
from geometry_msgs.msg import TransformStamped

TRANSLATION_X = 0.65
TRANSLATION_Y = 0.0
TRANSLATION_Z = -0.07

F_X= 607.3187866210938
F_Y = 607.1571044921875
C_X = 320.80078125
C_Y = 247.81671142578125

class EnvironmentModel(Node):

    def __init__(self, par='default'):
        super().__init__('environment_model')
        # create subscribers to /detectnet/detections, /odom and /camera/camera/aligned_depth_to_color/image_raw topics from Object Detection, Localization and Camer
        self.subscription = self.create_subscription(Detection2DArray, '/detectnet/detections', self.detections_callback, 10)
        self.subscription = self.create_subscription(Image, '/camera/camera/aligned_depth_to_color/image_raw', self.depth_callback, 10)
        # self.subscription = self.create_subscription(Image, '/camera/camera/depth/image_rect_raw', self.depth_callback, 10)
        self.subscription = self.create_subscription(Odometry, '/odom', self.odom_callback, 10)

        # create publisher to publish detected object position
        self.publish_objects_position = self.create_publisher(DetectedObjectsPositionArray, 'detected_objects_pos', 10)

        # initialize the transform broadcaster
        self.br = TransformBroadcaster(self)

```

```
45          # create timer of 0.02 sec to broadcast "v2x_frame"
46          self.timer = self.create_timer(0.02, self.broadcast_frame)
47
48          # initialize buffer and listen to the transforsm
49          self.tf_buffer = Buffer()
50          self.tf_listener = TransformListener(self.tf_buffer, self)
51
52          # intialize the variables
53          self.yaw = 0
54          self.flg_calc_xyz = 0
55          self.detection_time = 0
56          self.bbox = []
57          self.class_ids_confidence = []
58
59          # Log startup info
60          self.get_logger().info('✅ environment_model node started and ready!')
61
62      def detections_callback(self, data):
63          # store the detection time, centre coordinates and size of boundary box of the detected objects and their class ids and confidences
64          if self.flg_calc_xyz == 0: # store only when the coordinates are not calculated in the depth_callback function
65              self.detection_time = data.header.stamp.sec* 1e3 + data.header.stamp.nanosec * 1e-6
66              for item in data.detections:
67                  self.bbox.append([item.bbox.center.position.x, item.bbox.center.position.y, item.bbox.size_x, item.bbox.size_y])
68                  self.class_ids_confidence.append([item.results[0].hypothesis.class_id, int(item.results[0].hypothesis.score*100)])
69              if self.bbox == []:
70                  self.flg_calc_xyz = 0
71              else:
72                  self.flg_calc_xyz = 1
73
74      def depth_callback(self, data):
75
76          # exit if there are no detections
77          if self.bbox == []:
78              return
79
80          # extract step and depth image from depth topic
81          step = data.step
82          depth_image = np.frombuffer(data.data, dtype=np.uint16).reshape((data.height, data.width))
83
84          self.id = 0
85          dop = DetectedObjectsPositionArray()
86
87
88          for i in range(len(self.bbox)):
89              # calculate the top left and right bottom coordinates of the boundary box for each object
90              bbox_cx, bbox_cy, bbox_size_x, bbox_size_y= self.bbox[i][0], self.bbox[i][1], self.bbox[i][2], self.bbox[i][3]
91              x_min = max(0, int(bbox_cx - bbox_size_x / 2))
92              x_max = min(data.width, int(bbox_cx + bbox_size_x / 2))
93              y_min = max(0, int(bbox_cy - bbox_size_y / 2))
94              y_max = min(data.height, int(bbox_cy + bbox_size_y / 2))
95
```

```
96              # extract the depth values in the bounding box window and find the valid depth values (0.1 - 20m)
97              depth_window = depth_image[y_min:y_max, x_min:x_max] / 1000.0  #convert to meters
98              self.valid_depths = depth_window[(depth_window > 0.1) & (depth_window < 20.0)]
99
100             # skip the detected object is there are no valid depth values
101             if len(self.valid_depths) == 0:
102                 continue
103
104             # cluster the depth values using DBSCAN algorithm
105             depth_vals = self.valid_depths.reshape(-1, 1)
106             clustering = DBSCAN(eps=0.1, min_samples=30).fit(depth_vals)
107             self.labels = clustering.labels_
108
109             # choose the largest cluster and find the depth
110             if len(set(self.labels)) > 1:
111                 largest_cluster = np.argmax(np.bincount(self.labels[self.labels >= 0]))
112                 cluster_depths = depth_vals[self.labels == largest_cluster]
113                 self.dominant_depth = np.median(cluster_depths)
114             else:
115                 self.dominant_depth = np.median(self.valid_depths)
116
117             # calculate the coordinates in camera depth optical frame using depth
118             self.X = self.dominant_depth * (self.bbox[i][0] - C_X) / F_X
119             self.Y = self.dominant_depth * (self.bbox[i][1] - C_Y) / F_Y
120             self.Z = self.dominant_depth
121
122             #create point in camera depth optical frame
123             point_in_source = PointStamped()
124             point_in_source.header.stamp = self.get_clock().now().to_msg()
125             point_in_source.header.frame_id = 'camera_depth_optical_frame'  # example source frame
126             point_in_source.point.x = self.X
127             point_in_source.point.y = self.Y
128             point_in_source.point.z = self.Z
129
130             try:
131                 # lookup the transform from 'camera_depth_optical_frame' to 'v2x_frame'
132                 transform = self.tf_buffer.lookup_transform(
133                     'v2x_frame',   # target frame
134                     'camera_depth_optical_frame',   # source frame
135                     rclpy.time.Time())
136
137                 # transform the point to v2x_frame
138                 point_in_target = do_transform_point(point_in_source, transform)
139
140                 # store the position, class id and confidence of the detected object
141                 self.id = self.id + 1
142                 do = DetectedObject()
143                 do.id = self.id
144                 do.class_id,  do.confidence = self.class_ids_confidence[i][0], self.class_ids_confidence[i][1]
145                 do.x, do.y, do.z = int(point_in_target.point.x*100), int(point_in_target.point.y*100), int(point_in_target.point.z*100)
146                 dop.array.append(do)
```

```
147
148                 self.get_logger().info(
149                     f"Distance: {self.dominant_depth}: "
150                     f"Point in {point_in_source.header.frame_id}: "
151                     f"({point_in_source.point.x:.2f}, {point_in_source.point.y:.2f}) -->  "
152                     f"Point in {point_in_target.header.frame_id}: "
153                     f"({point_in_target.point.x:.2f}, {point_in_target.point.y:.2f})")
154
155             except TransformException as ex:
156                 self.get_logger().warn(f'Could not transform point: {ex}')
157
158         # publish the positions, class id and confidence of detected objects
159         dop.detection_time = self.detection_time
160         self.publish_objects_position.publish(dop)
161
162         # clear the array and variables
163         dop.array.clear()
164         self.bbox.clear()
165         self.flg_calc_xyz = 0
166
167     def broadcast_frame(self):
168         t = TransformStamped()
169         t.header.stamp = self.get_clock().now().to_msg()
170         t.header.frame_id = '4/base_link'
171         t.child_frame_id = 'v2x_frame'
172
173         # translation: center of v2x_frame relative to base link
174         t.transform.translation.x = TRANSLATION_X
175         t.transform.translation.y = TRANSLATION_Y
176         t.transform.translation.z = TRANSLATION_Z
177
178         # rotation: yaw only (roll, pitch = 0)
179         q = tf_transformations.quaternion_from_euler(0, 0, -self.yaw)
180         t.transform.rotation.x = q[0]
181         t.transform.rotation.y = q[1]
182         t.transform.rotation.z = q[2]
183         t.transform.rotation.w = q[3]
184
185         # broadcast the new frame
186         self.br.sendTransform(t)
187
188     def odom_callback(self, data):
189         q = data.pose.pose.orientation
190
191         # convert quaternion to euler to get yaw angle
192         _, _, self.yaw = tf_transformations.euler_from_quaternion([q.x, q.y, q.z, q.w])
193
194
195 def main(args=None):
196     rclpy.init(args=args)
197     node = EnvironmentModel()
```

```
198     rclpy.spin(node)
199     node.destroy_node()
200     rclpy.shutdown()
201
202
203 if __name__ == '__main__':
204     main()
```

« prev ^ index » next coverage.py v7.11.0, created at 2025-11-09 02:30 +0100